

David_计算几何

David_计算几何

2D计算几何

3D计算几何

2D计算几何

```
1 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
2 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
3 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
4 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
5 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
6 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
7
8 const double PI = acos(-1), eps = 1e-9;
9
10 int sgn(double x) {
11     if (fabs(x) < eps) return 0;
12     return x < 0 ? -1 : 1;
13 }
14
15 struct Pt{
16     double x, y;
17     Pt(double x = 0.0, double y = 0.0) : x(x), y(y) {}
18
19     Pt operator- (const Pt& b) const {return Pt(x - b.x, y - b.y);}
20
21     Pt operator+ (const Pt& b) const {return Pt(x + b.x, y + b.y);}
22
23     double operator% (const Pt& b) const {return x * b.y - y * b.x;}
24
25     Pt operator* (const double b) const {return Pt(x * b, y * b);}
26
27     double operator* (const Pt& b) const {return x * b.x + y * b.y;}
28
29     Pt operator/ (const double b) const {return Pt(x / b, y / b);}
30
31     bool operator< (const Pt& b) const {return x < b.x || (x == b.x && y < b.y);}
32
33     bool operator== (const Pt& b) const {return sgn(x - b.x) == 0 && sgn(y - b.y)
== 0;}
34
35     bool operator!= (const Pt& b) const {return sgn(x - b.x) != 0 || sgn(y - b.y)
!= 0;}
36 };
37
38 double dis(Pt a){
39     return sqrt(a * a);
40 }
41
42 double ddis(Pt a){
43     return a * a;
```

```

44 }
45
46 // 两向量夹角, 返回  $[0, \pi]$ , 不带方向
47 // !!! a 或 b 是零向量时会**静默返回 0**(而不是 nan), 需要自己先判
48 // 两向量夹角, 返回  $[0, \pi]$ , 不带方向
49 // !!! a 或 b 是零向量时会**静默返回 0**(而不是 nan), 需要自己先判
50 // 用 atan2 而不是 acos: acos 在接近  $\pm 1$  处丢精度,
51 // 老写法  $\text{acos}(a*b/|a|/|b|)$  对  $\text{Angle}((1,0),(1,1e-8))$  会直接返回 0
52 double Angle(Pt a, Pt b){
53     return atan2(fabs(a % b), a * b);
54 }
55
56 //在直线的哪边
57 // 点在直线上, 返回 0 (三点共线)
58 // 点在直线的逆时针方向, 返回 1
59 // 点在直线的顺时针方向, 返回 -1
60 //直线ab, 点c
61 // !!! 坐标绝对值必须  $\leq 9.4e7 (= \text{sqrt}(2^{53}))$ 
62 // 再大叉积就超出 double 的 53 位有效位, 会把不共线的三点判成共线:
63 //  $K = 1e8$  时  $A(0,0) B(K,K-1) C(K-1,K-2)$  的精确叉积是 -1, 这里算出来是 0,
64 // Andrew 凸包会因此丢掉真实顶点, 皮克定理跟着一起错
65 // 整数坐标且范围大时, 请单独用 long long / __int128 算叉积
66 int Cross(Pt a, Pt b, Pt c){
67     return sgn((b - a) % (c - a));
68 }
69
70
71 double cross(Pt a, Pt b, Pt c){
72     return (b - a) % (c - a);
73 }
74
75 struct Line {
76     Pt s, e;
77     double ang;
78     Line() {}
79     Line(Pt x, Pt y) : s(x), e(y) {
80         ang = atan2(e.y - s.y, e.x - s.x);
81     }
82
83     // 检查点 P 是否在向量 se 的右侧 (非法区域)
84     // 使用叉积判断:  $(e-s) \wedge (P-s) < 0$  则在右侧
85     bool onRight(const Pt& P) const {
86         return sgn((e - s) % (P - s)) < 0;
87     }
88
89     // 极角排序: 极角相同时, 把"更靠左"(即约束更强)的那条排在前面
90     // 去重只保留每组极角里的第一条, 所以这里必须是  $< 0$ 
91     // 写成  $> 0$  会保留约束最弱的那条, 半平面交的结果整体偏大(随机对拍 1963 组错 1910 组)
92     bool operator< (const Line& b) const {
93         if (sgn(ang - b.ang) != 0) return ang < b.ang;
94         return sgn((e - s) % (b.e - s)) < 0;
95     }
96 };
97
98
99 int Cross(Pt c, Line l){
100     Pt a = l.s, b = l.e;
101     return sgn((b - a) % (c - a));
102 }
103
104 //三点共线

```

```

105 bool In_one_line(Pt a, Pt b, Pt c){
106     return !sgn((b - a) % (c - a));
107 }
108 //点到直线距离
109 // a == b 时退化：不加保护会返回 nan，而 sgn(nan) == 1，会让 LWC 把"圆心处的退化直线"判成
    相离
110 double dist_ptl(Pt p, Pt a, Pt b){
111     if (a == b) return dis(p - a);
112     Pt v1 = p - a, v2 = b - a;
113     return fabs((v1 % v2) / dis(v2));
114 }
115 //点到线段距离
116 double dist_pts(Pt p, Pt a, Pt b){
117     if(a == b) return dis(p - a);
118     Pt v1 = b - a, v2 = p - a, v3 = p - b;
119     if(sgn(v1 * v2) < 0) return dis(p - a);
120     if(sgn(v1 * v3) > 0) return dis(p - b);
121     return fabs((v1 % v2) / dis(v1));
122 }
123 //点在线段上
124 // 注意 sgn 用的是绝对 eps，叉积量级是坐标的平方：坐标量级远小于 1 时会误判，
125 // 例如 OnSegment((5e-5, 5e-6), (0,0), (1e-4,0)) 返回 true，但那个点偏离了线段长度的 5%
126 bool OnSegment(Pt p, Pt a, Pt b){
127     Pt pa = a - p, pb = b - p;
128     return sgn(pa % pb) == 0 && sgn(pa * pb) <= 0;
129 }
130 //直线与线段相交：直线 ab，线段 cd
131 //(判断线段 cd 的两个端点是否分居直线 ab 两侧，参数顺序别写反)
132 bool Intersect_line_seg(Pt a, Pt b, Pt c, Pt d){
133     return Cross(a, b, c) * Cross(a, b, d) <= 0;
134 }
135 //线段与线段是否相交
136 bool Intersect_seg(Pt a, Pt b, Pt c, Pt d){
137     if (OnSegment(a, c, d) || OnSegment(b, c, d) || OnSegment(c, a, b) ||
    OnSegment(d, a, b)) return 1;
138     if (Cross(a, b, c) * Cross(a, b, d) >= 0) return 0;
139     if (Cross(c, d, a) * Cross(c, d, b) >= 0) return 0;
140     return 1;
141 }
142 //线段到线段距离
143 double dist_sts(Pt a, Pt b, Pt c, Pt d){
144     if(Intersect_seg(a, b, c, d)){
145         return 0.0;
146     }
147     return min({
148         dist_pts(a, c, d),
149         dist_pts(b, c, d),
150         dist_pts(c, a, b),
151         dist_pts(d, a, b),
152     });
153 }
154 //直线平行(两直线重合时也返回 true)
155 // 叉积必须除以两个方向向量的长度再和 eps 比，否则短线段会被误判：
156 // 长 1e-5 的水平线和长 1e-5 的竖直线互相垂直，但叉积只有 1e-10 < eps，不归一化就判成平行
157 bool Line_parallel(Line A, Line B){
158     Pt u = A.s - A.e, v = B.s - B.e;
159     double du = dis(u), dv = dis(v);
160     if (sgn(du) == 0 || sgn(dv) == 0) return true; // 退化的"直线"，当作平行
161     return sgn((u % v) / (du * dv)) == 0;
162 }
163 //直线交点（四个点）

```

```

164 Pt Inter_Line_Pt(Pt a, Pt b, Pt c, Pt d){
165     Pt u = b - a, v = d - c;
166     double t = ((a - c) % v) / (v % u);
167     return a + u * t;
168 }
169
170 // 直线交点: 输入两条 Line 对象 (方便半平面交调用)
171 Pt Inter_Line_Pt(Line a, Line b) {
172     Pt u = a.s - b.s;
173     Pt v1 = a.e - a.s;
174     Pt v2 = b.e - b.s;
175     double t = (v2 % u) / (v1 % v2);
176     return a.s + v1 * t;
177 }
178
179 // 半平面交
180 // 保留每一条直线的左侧区域
181 // !!!注意, 设置边界的时候, limit 一定要比 eps 大一点, 不然可能卡精度, 例如 limit=1e-11, eps=1e-
182 // !!!Line 只有 (Pt, Pt) 一个构造函数, 下面的例子别再传第三个参数
183 // !!!必须自己保证区域有界(先塞一圈大边界框), 否则返回的顶点里会混进 nan/inf 且不报错
184 // !!!下面这四行是**第二象限专用**的框(只框住 x<0 且 y>0), 是从某道旧题里留下来的,
185 // 当成通用边界框直接用会把可行域裁成空集。通用的框应该是:
186 // L.push_back(Line(Pt(-INF,-INF), Pt(INF,-INF)));
187 // L.push_back(Line(Pt(INF,-INF), Pt(INF, INF)));
188 // L.push_back(Line(Pt(INF, INF), Pt(-INF, INF)));
189 // L.push_back(Line(Pt(-INF, INF), Pt(-INF,-INF)));
190 // 第二象限版本(原样保留):
191 // L.push_back(Line(Pt(-INF, limit), Pt(-limit, limit)));
192 // L.push_back(Line(Pt(-limit, limit), Pt(-limit, INF)));
193 // L.push_back(Line(Pt(-limit, INF), Pt(-INF, INF)));
194 // L.push_back(Line(Pt(-INF, INF), Pt(-INF, limit)));
195 vector<Pt> getHalfPlaneIntersection(vector<Line>& L) {
196     if (L.empty()) return {}; // 不判空的话下面 ql[0] 越界, 直接段错误
197
198     // 1. 排序
199     sort(L.begin(), L.end());
200
201     // 2. 去重 (对于极角相同的直线, 只保留排序后第一条, 即最靠左的那条)
202     vector<Line> ql;
203     for (int i = 0; i < L.size(); i++) {
204         if (i == 0 || sgn(L[i].ang - L[i-1].ang) != 0) {
205             ql.push_back(L[i]);
206         }
207     }
208
209     // 3. 双端队列求解
210     deque<Line> dq;
211     deque<Pt> pts; // pts[i] 是 dq[i] 和 dq[i+1] 的交点
212
213     dq.push_back(ql[0]);
214     for (int i = 1; i < ql.size(); i++) {
215         // 如果队尾的交点在当前直线的右侧 (非法), 弹出队尾
216         while (!pts.empty() && ql[i].onRight(pts.back())) {
217             pts.pop_back();
218             dq.pop_back();
219         }
220         // 如果队首的交点在当前直线的右侧 (非法), 弹出队首
221         // 这一步是为了处理"环绕一圈后覆盖了起始点"的情况
222         while (!pts.empty() && ql[i].onRight(pts.front())) {
223             pts.pop_front();

```

```

224         dq.pop_front();
225     }
226
227     dq.push_back(q1[i]);
228     // 计算新加入直线与上一条直线的交点
229     if (dq.size() > 1) {
230         // 相邻两条直线方向平行(去重后只可能是极角差 pi 的反向平行)时,
231         // Inter_Line_Pt 会除以 0 得到 (inf, nan); 而 sgn(nan) == 1 使得 onRight
永远为 false,
232         // 这个脏点再也弹不出去, 最后混进结果里让 area() 变成 nan。
233         // 出现这种情况说明可行域无界或为空, 直接判无解
234         Line& u = dq[dq.size() - 2];
235         Line& v = dq.back();
236         if (sgn((u.e - u.s) % (v.e - v.s)) == 0) return {};
237         pts.push_back(Inter_Line_Pt(u, v));
238     }
239 }
240
241 // 4. 收尾检查: 队首的直线可能会切掉队尾的点
242 while (!pts.empty() && dq.front().onRight(pts.back())) {
243     pts.pop_back();
244     dq.pop_back();
245 }
246
247 // 如果队列中直线少于3条, 无法构成多边形
248 if (dq.size() < 3) return {};
249
250 // 5. 计算首尾交点, 封闭多边形
251 // 同样要防反向平行: 三条直线 y>=0 / x<=1 / y<=1 会走到这里, dq.size() == 3 躲过上面
的判断,
252 // 首尾两条极角差 pi, 不拦的话返回的第三个顶点是 (inf, nan)
253 {
254     Line& u = dq.back();
255     Line& v = dq.front();
256     if (sgn((u.e - u.s) % (v.e - v.s)) == 0) return {};
257 }
258 pts.push_back(Inter_Line_Pt(dq.back(), dq.front()));
259
260 // 将 deque 转为 vector 返回
261 vector<Pt> res;
262 for (auto p : pts) res.push_back(p);
263 return res;
264 }
265
266 // ----- 多边形 -----
267 // 三角形面积
268 double Triangle_area(Pt A, Pt B, Pt C){
269     return fabs((B - A) % (C - A)) / 2;
270 }
271 //凸多边形面积
272 // 因为叉积求得的三角形面积是有向的, 在外面的面积可以正负抵消掉
273 // 所以能够求任意多边形面积(凸, !凸)
274 // !!! 但必须是**简单多边形**(边不自交)。自交多边形会正负相消, 比如蝴蝶结形状返回 0
275 // p[]下标从 0 开始, 长度为 n
276 double area(vector<Pt>& p){
277     int n = p.size();
278     double s = 0;
279     for(int i = 1; i <= n - 2; i++){
280         s += (p[i] - p[0]) % (p[i + 1] - p[0]);
281     }
282     return fabs(s / 2); // 无向面积

```

```

283 //return S / 2;//有向面积
284 }
285
286 //点在多边形内(射线法)
287 // 适用于任意简单多边形，顺时针逆时针给出都可以
288 // 注意：内部要除以一条边两端点的 y 差，近水平且 x 跨度极大的边仍会掉精度，
289 // 不是完全不用管误差(整数坐标下随机 6 万次查询无误)
290 // 点在多边形边上，返回 -1
291 // 点在多边形内，返回 1
292 // 点在多边形外，返回 0
293
294 // p[] 的下标从 0 开始，长度为 n
295 int InPolygon(Pt P, vector<Pt>& p) {
296     int n = p.size();
297     bool flag = false;
298     for (int i = 0, j = n - 1; i < n; j = i++) {
299         Pt p1 = p[i], p2 = p[j];
300         if (OnSegment(P, p1, p2)) return -1;
301         if (sgn(P.y - p1.y) > 0 == sgn(P.y - p2.y) > 0) continue;
302         if (sgn((P.y - p1.y) * (p1.x - p2.x) / (p1.y - p2.y) + p1.x - P.x) > 0)
303             flag = !flag;
304     }
305     return flag;
306 }
307 //是否为凸多边形 (convex; 函数名 Is_convex 是历史拼写错误，搜 convex 搜不到它)
308 // !!! 只检查拐向是否一致，**自交多边形会被判成凸**：五角星实测返回 true
309 // 调用前必须自己保证是简单多边形；全部顶点共线的退化多边形也返回 true
310 bool Is_convex(vector<Pt>& p){
311     int n = p.size();
312     if (n < 3) return false; // n == 1 时下面的 k = n - 2 = -1，会越界读 p[-1]
313     bool s[3] = {0, 0, 0};
314     for (int i = 0, j = n - 1, k = n - 2; i < n; k = j, j = i++) {
315         int cnt = sgn((p[i] - p[j]) % (p[k] - p[j])) + 1;
316         s[cnt] = true;
317         if (s[0] && s[2]) return false;
318     }
319     return true;
320 }
321
322
323 // ----- 圆 -----
324 struct Circle{
325     Pt o;
326     double r;
327     Circle(Pt o = Pt(), double r = 0) : o(o), r(r) {}
328
329     double S(){return PI * r * r;}
330     double C(){return PI * 2 * r;}
331 };
332 //扇形面积(有向，逆时针为正)
333 // !!! A、B 必须是**相对圆心**的向量。圆心不在原点时要写 SectorArea(A - c.o, B - c.o,
334 // c.r)
335 // 直接传绝对坐标不会报错，但结果完全是错的(实测圆心(10,10)、R=5 的 90 度扇形
336 // 正确值 19.635，传绝对坐标得到 4.935)
337 // !!! 只对夹角 <= π 的扇形成立(Angle 只返回 [0, π])，优角要自己拆成两块加起来
338 double SectorArea(Pt A, Pt B, double R){
339     double angle = Angle(A, B);
340     if(sgn(A % B) < 0) angle = -angle;
341     return R * R * angle / 2;
342 }
343 //点和圆的位置关系

```

```

343 // 点在圆上, 返回 0
344 // 点在圆外, 返回 -1
345 // 点在圆内, 返回 1
346 int PWC(Pt p, Circle c){
347     double d = dis(p - c.o);
348     if(sgn(d - c.r) == 0) return 0;
349     if(sgn(d - c.r) > 0) return -1;
350     return 1;
351 }
352 //直线和圆的位置关系
353 // 相切, 返回 0
354 // 相交, 返回 1
355 // 相离, 返回 -1
356 int LWC(Pt A, Pt B, Circle c) {
357     double d = dist_ptl(c.o, A, B);
358     if (sgn(d - c.r) == 0) return 0;
359     if (sgn(d - c.r) > 0) return -1;
360     return 1;
361 }
362
363 //直线和圆的交点
364 // 判据必须和 LWC 一致, 都比"未平方"的距离。
365 // 直接比 d2 与 r*r 的话, 半径小于约 3.2e-5 时 |d2 - r*r| 恒小于 eps, 所有相交都被判成相切:
366 // LWC 说相交(返回 1), 这里却只返回 1 个点, if(LWC(..)==1){ 用 v[0], v[1]; } 会越界读 v[1]
367 vector<Pt> Intersection_line_circle(Pt A, Pt B, Circle c) {
368     if (A == B) return {}; // 退化, 两点重合构不成直线
369     Pt AB = B - A;
370     double len2 = AB * AB;
371     Pt pr = A + AB * ((c.o - A) * AB / len2);
372     double d2 = ddis(pr - c.o);
373
374     int status = sgn(sqrt(d2) - c.r);
375     if (status > 0) return {}; // 相离, 返回空 vector
376
377     double base = sqrt(max(0.0, c.r * c.r - d2));
378     if (status == 0) return {pr}; // 相切, 返回 1 个点
379
380     // 相交, 返回 2 个点
381     Pt e = AB / sqrt(len2);
382     return {pr + e * base, pr - e * base};
383 }
384 //圆与圆的位置关系
385 // 相离, 返回 -1
386 // 外切, 返回 0
387 // 内切(A 包含 B), 返回 1
388 // 内切(B 包含 A), 返回 2
389 // 内含(A 包含 B), 返回 3
390 // 内含(B 包含 A), 返回 4
391 // 相交, 返回 5
392 // !!! 两圆完全重合(圆心半径都相同)时返回 1(内切), 没有单独的"重合"返回值, 需要自己先判
393
394 int Circle_with_circle(Circle A, Circle B) {
395     double len1 = dis(A.o - B.o);
396     double len2 = A.r + B.r;
397     if (sgn(len1 - len2) > 0) return -1;
398     if (sgn(len1 - len2) == 0) return 0;
399     if (sgn(len1 + len2 - 2 * A.r) == 0) return 1;
400     if (sgn(len1 + len2 - 2 * B.r) == 0) return 2;
401     if (sgn(len1 + len2 - 2 * A.r) < 0) return 3;

```

```

402     if (sgn(len1 + len2 - 2 * B.r) < 0) return 4;
403     return 5;
404 }
405
406 //圆与圆的交点
407 // 相交，返回 2 个点
408 // 相切，返回 1 个点(不是两个重复的点，用 .size() 区分)
409 // 相离/内含，返回空 vector
410
411 // 函数内部已经判过位置关系，可以直接调用
412 vector<Pt> Intersection_circle_circle(Circle A, Circle B) {
413     double d = dis(A.o - B.o);
414     // 圆心重合：半径相同是无穷多交点，半径不同是无交点，统一返回空
415     // 不加这句话，两圆完全重合时会走到下面的 (B.o - A.o) / d 除以 0，返回 (nan, nan)
416     if (sgn(d) == 0) return {};
417     // 情况 1：相离或内含（无交点）
418     if (sgn(d - (A.r + B.r)) > 0 || sgn(d - fabs(A.r - B.r)) < 0) {
419         return {};
420     }
421
422     // 情况 2：相切（1 个交点）
423     if (sgn(d - (A.r + B.r)) == 0 || sgn(d - fabs(A.r - B.r)) == 0) {
424         Pt e = (B.o - A.o) / d;
425         if (sgn(d - (A.r + B.r)) == 0) return {A.o + e * A.r}; // 外切：切点在两圆心
426         // 内切：切点在远离大圆圆心的那一侧，不能统一用 A.o + e*A.r
427         // 例：A(36.5,0,r=0.4) 内切于 B(42.1,0,r=6) 时，切点应是 (36.1,0) 而非
428         // (36.9,0)
429         if (sgn(A.r - B.r) > 0) return {A.o + e * A.r}; // A 含 B
430         return {A.o - e * A.r}; // B 含 A
431     }
432
433     // 情况 3：相交（2 个交点）
434     double a = acos(max(-1.0, min(1.0, (A.r * A.r + d * d - B.r * B.r) / (2.0 *
435     A.r * d)))));
436     double t = atan2(B.o.y - A.o.y, B.o.x - A.o.x);
437     return {
438         A.o + Pt(A.r * cos(t + a), A.r * sin(t + a)),
439         A.o + Pt(A.r * cos(t - a), A.r * sin(t - a))
440     };
441 }
442 //求圆外一点对圆的两个切点
443 vector<Pt> TangentPt_Pt_circle(Pt p, Circle c) {
444     double d2 = ddis(p - c.o);
445     int status = sgn(d2 - c.r * c.r);
446
447     if (status < 0) return {}; // 点在圆内，无切点
448     if (status == 0) return {p}; // 点在圆上，切点就是自身
449
450     // 点在圆外，2 个切点
451     double d = sqrt(d2);
452     double l = sqrt(d2 - c.r * c.r);
453     double angle = asin(max(-1.0, min(1.0, c.r / d)));
454     Pt e = (c.o - p) / d;
455
456     // 旋转向量得到切点方向
457     auto rotate = [](Pt v, double ang) {
458         return Pt(v.x * cos(ang) - v.y * sin(ang), v.x * sin(ang) + v.y *
459         cos(ang));
460     };
461 }

```



```

459     return {
460         p + rotate(e, angle) * l,
461         p + rotate(e, -angle) * l
462     };
463 }
464
465 //求三角形外接圆
466 // !!! 前置条件: A、B、C 不共线。共线时 D = 0, 返回的圆心是 (-inf, inf)、半径 inf, 没有任何提示
467 Circle get_circumcircle(Pt A, Pt B, Pt C) {
468     double Bx = B.x - A.x, By = B.y - A.y;
469     double Cx = C.x - A.x, Cy = C.y - A.y;
470     double D = 2 * (Bx * Cy - By * Cx);
471
472     double x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
473     double y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
474     Pt P(x, y);
475     return Circle(P, dis(A - P));
476 }
477 //三角形内切圆
478 Circle get_incircle(Pt A, Pt B, Pt C) {
479     double a = dis(B - C);
480     double b = dis(A - C);
481     double c = dis(A - B);
482     Pt p = (A * a + B * b + C * c) / (a + b + c);
483     return Circle(p, dist_ptl(p, A, B));
484 }
485 // 要保证传入的点是整点
486 // 线段上的整点个数
487 // 注意按照要求修改返回值
488 // 端点相同时(退化成一个点) gcd(0, 0) = 0, "包含端点" 版本返回 1, 是对的
489 // 但改用下面"不包含端点"那一行时, 退化线段会返回 -1, 得自己特判
490 int IntegerPt_on_seg(Pt A, Pt B) {
491     int x = abs(llround(A.x) - llround(B.x));
492     int y = abs(llround(A.y) - llround(B.y));
493     return __gcd(x, y) + 1; // 包含端点
494     return __gcd(x, y) - 1; // 不包含端点
495 }
496 // 返回多边形边**上**整点的个数(含顶点)
497 // 点需要是顺时针(逆时针)给出
498
499 // p[] 下标从 0 开始, 长度为 n
500 // 记得 #define int long long: res 会累加到 n * 2e9 量级, 32 位 int 直接溢出
501 int IntegerPt_on_polygon(vector<Pt>& p) {
502     int n = p.size();
503     int res = 0;
504     for (int i = 0, j = n - 1; i < n; j = i++) {
505         int x = abs(llround(p[i].x) - llround(p[j].x));
506         int y = abs(llround(p[i].y) - llround(p[j].y));
507         res += __gcd(x, y); // 每条边只算一个端点, 绕一圈刚好不重不漏
508     }
509     return res;
510 }
511
512 // ----- 皮克定理 (Pick's Theorem) -----
513 // S = a + b / 2 - 1
514 //      S : 多边形面积      a : 内部整点数      b : 边界整点数(含顶点)
515 // 前提: 所有顶点都是整点, 且是简单多边形(边不自交)
516 //      凸多边形(比如 Andrew 求出来的凸包)天然满足;
517 //      凸包把边上的共线点删掉也不影响 b, 因为 gcd 是按整条边算的
518 // 注意: 千万不要拿 double 的 area() 直接套公式, 1e-9 的误差取整后整体就差 1

```

```

519 // 下面统一用 "两倍面积" 走纯整数运算(记得 #define int long long)
520 // 用法: vector<Pt> hull = Andrew(p); int cnt = IntegerPt_polygon(hull);
521
522 // 多边形的两倍面积(顶点是整点时一定是整数), 顺/逆时针给出都可以
523 // 坐标绝对值超过 1e9 时把 int 换成 __int128 防溢出
524 int area2(vector<Pt>& p) {
525     int n = p.size();
526     int s = 0;
527     for (int i = 0, j = n - 1; i < n; j = i++) {
528         int x1 = llround(p[j].x), y1 = llround(p[j].y);
529         int x2 = llround(p[i].x), y2 = llround(p[i].y);
530         s += x1 * y2 - y1 * x2;
531     }
532     return abs(s);
533 }
534
535 // 返回不包括边界的, 多边形**内**整点个数
536 //  $a = s - b / 2 + 1 = (2S - b + 2) / 2$ , 分子恒为偶数, 整除不掉精度
537 int IntegerPt_in_polygon(vector<Pt>& p) {
538     return (area2(p) - IntegerPt_on_polygon(p) + 2) / 2;
539 }
540
541 // 返回多边形整点总数(内部 + 边界)
542 //  $a + b = (2S + b + 2) / 2$ 
543 // 退化情况(所有点共线, 面积为 0): 本函数返回的仍是那条线段上的整点数,
544 // 但 IntegerPt_in_polygon 会返回 0 或负数, 需要自行特判
545 int IntegerPt_polygon(vector<Pt>& p) {
546     return (area2(p) + IntegerPt_on_polygon(p) + 2) / 2;
547 }
548
549 int getQuadrant(Pt p, Pt center) {
550     double x = p.x - center.x;
551     double y = p.y - center.y;
552     if (x > 0 && y >= 0) return 1; // 第一象限
553     if (x <= 0 && y > 0) return 2; // 第二象限
554     if (x < 0 && y <= 0) return 3; // 第三象限
555     if (x >= 0 && y < 0) return 4; // 第四象限
556     return 0;
557 }
558
559 // !!! center 必须按值传, 不能按引用。polarSort(p, p[0]) 这种写法下 sort 会移动 p[0],
560 // 按引用捕获的 center 排到一半就变了, 比较函数自相矛盾 -> 结果错乱 + std::sort UB
561 // (实测 n = 30 就能和"绕副本排序"给出不同结果)
562 void polarSort(vector<Pt>& Pts, Pt center) {
563     sort(Pts.begin(), Pts.end(), [center](const Pt& a, const Pt& b) {
564         int quadA = getQuadrant(a, center);
565         int quadB = getQuadrant(b, center);
566
567         if (quadA != quadB) {
568             return quadA < quadB;
569         }
570
571         // 同一象限内使用叉积
572         // 这里必须用精确的 != 0 而不是 fabs(cp) > eps:
573         // eps 版本的"等价"不满足传递性(a~b 且 b~c 但 a<c), 是 std::sort 的 UB
574         double cp = (a - center) % (b - center);
575         if (cp != 0) {
576             return cp > 0; // 逆时针排序
577         }
578
579         // 共线时, 距离近的在前

```

```

580         return ddis(a - center) < ddis(b - center);
581     });
582 }
583
584 //凸包算法
585 // 返回逆时针序，边上的共线点会被删掉(只留角点)
586 // !!! 会就地 sort 传入的 p，打乱调用方的数组；要保留原顺序就先自己复制一份
587 // !!! 坐标绝对值 > 9.4e7 时 Cross 会失真丢顶点，见 Cross 处的说明
588 // 退化情况：所有点共线或重合时返回 2 个点，0/1 个点原样返回
589 vector<Pt> Andrew(vector<Pt>& p){
590     sort(p.begin(), p.end());
591     int n = p.size();
592     vector<Pt> ans;
593     for(int i = 0; i < n; i++){
594         while(ans.size() >= 2 && Cross(ans[ans.size() - 2], ans.back(), p[i]) <=
0){
595             ans.pop_back();
596         }
597         ans.push_back(p[i]);
598     }
599
600     int t = ans.size();
601     for(int i = n - 2; i >= 0; i--){
602         while(ans.size() > t && Cross(ans[ans.size() - 2], ans.back(), p[i]) <=
0){
603             ans.pop_back();
604         }
605         ans.push_back(p[i]);
606     }
607
608     if(ans.size() > 1) ans.pop_back();
609     return ans;
610 }
611 }
612
613 int norm_idx(int x, int n){
614     x %= n;
615     if(x < 0) x += n;
616     return x;
617 }
618
619 // 点在已排序凸包中的位置：
620 // 返回 -1 表示外部，0 表示边界上，1 表示内部
621 // 要求 hull 为逆时针排序的凸包
622 int InConvex(Pt q, const vector<Pt>& hull){
623     int n = hull.size();
624     if(n == 0) return -1;
625     if(n == 1) return q == hull[0] ? 0 : -1;
626     if(n == 2) return OnSegment(q, hull[0], hull[1]) ? 0 : -1;
627
628     int s1 = Cross(hull[0], hull[1], q);
629     int s2 = Cross(hull[0], hull[n - 1], q);
630     if(s1 < 0 || s2 > 0) return -1;
631     if(OnSegment(q, hull[0], hull[1]) || OnSegment(q, hull[0], hull[n - 1]))
return 0;
632
633     int l = 1, r = n - 1;
634     while(l + 1 < r){
635         int mid = (l + r) >> 1;
636         if(Cross(hull[0], hull[mid], q) >= 0) l = mid;
637         else r = mid;

```

```

638     }
639
640     int side = Cross(hull[l], hull[(l + 1) % n], q);
641     if(side < 0) return -1;
642     if(OnSegment(q, hull[l], hull[(l + 1) % n])) return 0;
643     return side == 0 ? 0 : 1;
644 }
645
646 // 若 q 在凸包外部, 返回一个可见边的编号 i, 表示边 hull[i] -> hull[i+1] 对 q 可见
647 // 若 q 在凸包内部或边界上, 返回 -1
648 int FindVisibleEdge(Pt q, const vector<Pt>& hull){
649     int n = hull.size();
650     if(n < 3) return -1;
651
652     if(Cross(hull[0], hull[1], q) < 0) return 0;
653     if(Cross(hull[0], hull[n - 1], q) > 0) return n - 1;
654
655     int l = 1, r = n - 1;
656     while(l + 1 < r){
657         int mid = (l + r) >> 1;
658         if(Cross(hull[0], hull[mid], q) >= 0) l = mid;
659         else r = mid;
660     }
661
662     return Cross(hull[l], hull[(l + 1) % n], q) < 0 ? l : -1;
663 }
664
665 bool VisibleEdge(Pt q, const vector<Pt>& hull, int idx){
666     int n = hull.size();
667     return Cross(hull[norm_idx(idx, n)], hull[norm_idx(idx + 1, n)], q) < 0;
668 }
669
670 // 返回外点 q 到已排序凸包的两个切点下标
671 // 返回值中的 first, second 分别是可见边链的起点和终点, 按凸包逆时针顺序给出
672 // 若 q 不在凸包外部, 则返回 {-1, -1}
673 pair<int, int> TangentIndexConvex(Pt q, const vector<Pt>& hull){
674     int n = hull.size();
675     if(n == 0) return {-1, -1};
676     if(n == 1) return {0, 0};
677     if(n == 2) return {0, 1};
678
679     int k = FindVisibleEdge(q, hull);
680     if(k == -1) return {-1, -1};
681
682     // 从 k 向两侧线性扩展可见边链(凸包上可见边是一段连续弧)
683     // 不能用二分: 链可能从 0 号边开始或绕过 0 号边,
684     // 此时 "第 k±mid 条边可见" 不是 mid 的单调函数, 二分会返回错误的切点
685     // (实测: 可见链为 0..3 号边时, 原二分把左边切点从 0 错成 1)
686     int right_len = 0;
687     while(VisibleEdge(q, hull, k + right_len + 1)) right_len++;
688     int left_len = 0;
689     while(VisibleEdge(q, hull, k - left_len - 1)) left_len++;
690
691     int left_edge = norm_idx(k - left_len, n);
692     int right_edge = norm_idx(k + right_len, n);
693     return {left_edge, norm_idx(right_edge + 1, n)};
694 }
695
696 pair<Pt, Pt> TangentPointConvex(Pt q, const vector<Pt>& hull){
697     auto idx = TangentIndexConvex(q, hull);
698     if(idx.first == -1) return {Pt(), Pt()};

```

```

699     return {hull[idx.first], hull[idx.second]};
700 }
701
702 bool InTangentCone(Pt a, Pt b, const vector<Pt>& hull){
703     auto idx = TangentIndexConvex(a, hull);
704     if(idx.first == -1) return InConvex(a, hull) >= 0;
705     return Cross(a, hull[idx.first], b) <= 0 && Cross(a, hull[idx.second], b) >=
0;
706 }
707
708 // 判断线段 ab 是否与已排序凸包相交
709 // 要求 hull 为逆时针排序的凸包
710 bool Intersect_seg_convex(Pt a, Pt b, const vector<Pt>& hull){
711     int n = hull.size();
712     if(n == 0) return false;
713     if(n == 1) return OnSegment(hull[0], a, b);
714     if(n == 2) return Intersect_seg(a, b, hull[0], hull[1]);
715
716     if(InConvex(a, hull) >= 0 || InConvex(b, hull) >= 0) return true;
717     return InTangentCone(a, b, hull) && InTangentCone(b, a, hull);
718 }

```

3D计算几何

```

1  const double PI = acos(-1);
2
3  int sgn(double x) {
4      if (fabs(x) < 1e-8) return 0;
5      return x < 0 ? -1 : 1;
6  }
7
8  struct Point{
9      double x, y, z;
10     Point(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
11
12     Point operator+ (const Point& B) const {return Point(x + B.x, y + B.y, z +
B.z);}
13
14     Point operator- (const Point& B) const {return Point(x - B.x, y - B.y, z -
B.z);}
15     //点乘
16     double operator* (const Point& B) const {return x * B.x + y * B.y + z * B.z;}
17     //叉乘
18     Point operator% (const Point& B) const {return Point(y * B.z - z * B.y, z *
B.x - x * B.z, x * B.y - y * B.x);}
19     //数乘
20     Point operator* (const double& B) const {return Point(x * B, y * B, z * B);}
21     //数除 (
22     Point operator/ (const double& B) const {return Point(x / B, y / B, z / B);}
23
24     bool operator== (const Point& B) const {return sgn(x - B.x) == 0 && sgn(y -
B.y) == 0 && sgn(z - B.z) == 0;}
25
26     bool operator!= (const Point& B) const {return sgn(x - B.x) != 0 || sgn(y -
B.y) != 0 || sgn(z - B.z) != 0;}
27
28 };
29
30
31 double dis(Point& a){

```

```
32     return sqrt(a * a);
33 }
34
35 double ddis(Point& a){
36     return a * a;
37 }
38
39 // 两点间距离
40 double dis(Point& a, Point& b){
41     return sqrt((a - b) * (a - b));
42 }
43 //两点间距离平方
44 double ddis(Point& a, Point& b){
45     return (a - b) * (a - b);
46 }
47 //向量夹角
48 double Angle(Point& a, Point& b){
49     return acos(a * b / dis(a) / dis(b));
50 }
```